Universidad Tecnológica del Perú

DISEÑO DE PATRONES

DOCENTE:
Mg. Iván Robles Fernández

MOTIVACIÓN

- Hagan grupos de 4 o 5 personas
- Piensen en 3 palabras que describan sus intereses o preferencias (Ejm: música, deportes, viajar)
- Compartan sus palabras con su equipo y expliquen por qué las eligieron.
- ¿Cuáles son las similitudes o diferencias que encontraron entre ustedes?

🕐 **5 minutos**

Universidad Tecnológica del Perú

CONOCIMIENTOS PREVIOS

¿Qué entiendes por principios de sustitución de Liskov?

LOGRO DE LA SESIÓN

Al finalizar la unidad el participante identifica los principios de diseño de los patrones de software para la solución de problemas

IMPORTANCIA DE LA SESIÓN

La aplicación de los patrones de software a permitirá solucionar los problemas habituales de software planteados.

TEMA :

- Principio de sustitución de Liskov.
- Principio Abierto/Cerrado.

Principio de sustitución de Liskov (LSP)

Es el tercer principios SOLID en la programación orientada a objetos. Este principio fue formulado por Barbara Liskov y establece que:

"Los objetos de una clase derivada deben ser reemplazables por objetos de la clase base sin alterar el funcionamiento del programa.“

En otras palabras, si una clase S es una subclase de T, entonces los objetos de T deben poder ser sustituidos por objetos de S sin que el comportamiento del programa se vea afectado.

Principio de sustitución de Liskov (LSP)

Importancia

- **Mejora la mantenibilidad del código**: Facilita la extensión y modificación del software sin introducir errores.
- **Promueve la reutilización del código**: Las subclases pueden usarse en cualquier lugar donde se use la clase base.
- **Aumenta la robustez del software**: Ayuda a crear sistemas más fiables y menos propensos a fallos.

Principio de sustitución de Liskov (LSP)

Ventajas

- **Flexibilidad y Extensibilidad**: Permite añadir nuevas funcionalidades sin modificar el código existente.

- **Reducción de Errores**: Al cumplir con LSP, se reducen las probabilidades de errores al sustituir subclases por clases base.

Principio de sustitución de Liskov (LSP)

Desventajas

- **Mayor Complejidad Inicial**: Requiere una planificación cuidadosa del diseño de clases.
- **Sobrecarga de Diseño:** En algunos casos, puede ser excesivo para aplicaciones simples.

Principio de sustitución de Liskov (LSP)

Ejemplo: Imaginemos una jerarquía de clases relacionada con aves:

```
class Ave {
    public void volar() {
        System.out.println("El ave está volando");
    }
}

class Pinguino extends Ave {
    @Override
    public void volar() {
        throw new UnsupportedOperationException("Los pingüinos no pueden volar");
    }
}
```

Como se aplicaría el Principio de LSP

Principio de sustitución de Liskov (LSP)

Ejemplo: Imaginemos una jerarquía de clases relacionada con aves:

Aquí, tenemos una clase base Ave y una subclase Pinguino.

Según el principio LSP, un Pinguino debería poder sustituir a un Ave sin problemas.

Sin embargo, en este caso, si intentamos llamar al método volar() en un objeto Pinguino, obtendremos una excepción, lo cual rompe el principio LSP.

Principio de sustitución de Liskov (LSP)

Solución: Para cumplir con el LSP, podemos reestructurar nuestras clases de la siguiente manera:

```
class Ave {
    // Métodos comunes para todas las aves
}

class AveVoladora extends Ave {
    public void volar() {
        System.out.println("El ave está volando");
    }
}

class Pinguino extends Ave {
    // Métodos específicos para el pingüino
}
```

Aplicacion del Principio de LSP

De esta forma, tenemos una clase Ave que no asume que todas las aves pueden volar, y una subclase AveVoladora que añade el comportamiento de volar. Ahora, Pinguino puede ser sustituido por Ave sin problemas.

Principio de sustitución de Liskov (LSP)

Ejercicio Practico:

Supongamos que tenemos una clase base Vehiculo y dos subclases Coche y Bicicleta. Queremos asegurarnos de que la subclase Bicicleta cumple con el principio LSP.

Principio de sustitución de Liskov (LSP)

Ejercicio Practico:

```
class Vehiculo {
    public void arrancar() {
        System.out.println("El vehículo ha arrancado");
    }

    public void detener() {
        System.out.println("El vehículo se ha detenido");
    }
}

class Coche extends Vehiculo {
    public void abrirPuertas() {
        System.out.println("Las puertas del coche están
abiertas");
    }
}
```

```
class Bicicleta extends Vehiculo {
    @Override
    public void arrancar() {
        System.out.println("La bicicleta ha
comenzado a moverse");
    }

    @Override
    public void detener() {
        System.out.println("La bicicleta se ha
detenido");
    }
}
```

1. ¿Cumple Bicicleta con el principio LSP?
2. ¿Qué cambios harías para mejorar la estructura de clases y cumplir con LSP?

Principio de sustitución de Liskov (LSP)

Solucion

1. Sí, **Bicicleta** cumple con el LSP porque puede sustituir a **Vehiculo** sin problemas.
   La clase Bicicleta simplemente redefine el comportamiento de **arrancar y detener** sin romper la funcionalidad esperada.

1. No se requieren cambios para cumplir con LSP, ya que **Bicicleta** está bien diseñada. Sin embargo, podríamos considerar agregar métodos específicos para **Bicicleta y Coche** en sus propias clases para reflejar comportamientos únicos.

Principio Abierto/Cerrado

Es uno de los principios SOLID en la programación orientada a objetos. Fue formulado por Bertrand Meyer y establece que:

"El software debe estar abierto para la extensión, pero cerrado para la modificación.“

Esto significa que debemos diseñar nuestras clases y módulos de tal manera que puedan ser extendidos con nuevas funcionalidades sin necesidad de modificar el código existente.

Principio Abierto/Cerrado
Importancia

- **Facilita la Mantenibilidad**: Reduce la necesidad de cambiar el código existente, lo que minimiza la introducción de errores.
- **Fomenta la Reutilización**: Permite añadir nuevas características a los componentes existentes de manera más sencilla.
- **Aumenta la Flexibilidad**: Los sistemas se vuelven más adaptables a los cambios y mejoras.

Principio Abierto/Cerrado
Ventajas

- **Reducción de Errores:** Minimiza los cambios en el código existente, lo que reduce la posibilidad de introducir nuevos errores.
- **Evolución del Software:** Permite que el software evolucione de manera más controlada y segura.
- **Adaptabilidad:** Hace que el sistema sea más fácil de adaptar a nuevos requerimientos.

Principio Abierto/Cerrado
Desventajas

- **Complejidad Inicial:** Implementar OCP puede requerir una planificación y diseño más cuidadosos desde el inicio.
- **Sobrecarga de Código:** A veces puede llevar a una sobrecarga de clases e interfaces adicionales, aumentando la complejidad del sistema.

Principio Abierto/Cerrado

Ejemplo: Imaginemos que tenemos un sistema de cálculo de salario para diferentes tipos de empleados.

```
class Empleado {
    public double calcularSalario() {
        // Implementación básica
        return 0;
    }
}

class EmpleadoTiempoCompleto extends
Empleado {
    @Override
    public double calcularSalario() {
        return 3000;
    }
}
```

```
class EmpleadoMedioTiempo extends Empleado {
    @Override
    public double calcularSalario() {
        return 1500;
    }
}
```

Como se aplicaría el Principio de Abierto/Cerrado

Principio Abierto/Cerrado

Ejemplo:

Si queremos añadir un nuevo tipo de empleado, por ejemplo, un contratista, siguiendo el OCP, deberíamos hacerlo sin modificar las clases existentes:

```
class EmpleadoContratista extends
Empleado {
    @Override
    public double calcularSalario() {
        return 2000;
    }
}
```

Universidad Tecnológica del Perú

Principio Abierto/Cerrado

Solucion: Para cumplir mejor con el OCP, podemos usar interfaces o clases abstractas para definir el comportamiento común y permitir la extensión:

```
interface Empleado {
    double calcularSalario();
}

class EmpleadoTiempoCompleto implements Empleado {
    @Override
    public double calcularSalario() {
        return 3000;
    }
}

class EmpleadoMedioTiempo implements Empleado {
    @Override
    public double calcularSalario() {
        return 1500;
    }
}
```

```
class EmpleadoContratista implements Empleado {
    @Override
    public double calcularSalario() {
        return 2000;
    }
}
```

De esta manera, podemos agregar nuevos tipos de empleados sin modificar las clases existentes.

Principio Abierto/Cerrado

Ejercicio Practico: Supongamos que tenemos un sistema de notificaciones que puede enviar mensajes a través de diferentes canales (email, SMS). Queremos asegurarnos de que el sistema cumple con el OCP.

```
class Notificador {
    public void enviarNotificacion(String mensaje) {
        // Enviar notificación por defecto (por ejemplo, email)
    }
}

class NotificadorSMS extends Notificador {
    @Override
    public void enviarNotificacion(String mensaje) {
        // Enviar notificación por SMS
    }
}
```

1. ¿Cumple Bicicleta con el principio LSP?
2. ¿Qué cambios harías para mejorar la estructura de clases y cumplir con LSP?

Principio Abierto/Cerrado

Solución:

No, Notificador no cumple completamente con el OCP, ya que cada vez que queramos añadir un nuevo canal de notificación, tendríamos que modificar la clase Notificador.

Podemos usar una interfaz para Notificador y crear implementaciones específicas para cada canal de notificación:

Principio Abierto/Cerrado

Solución:

```
interface Notificador {
    void enviarNotificacion(String mensaje);
}

class NotificadorEmail implements
Notificador {
    @Override
    public void enviarNotificacion(String
mensaje) {
        // Enviar notificación por email
    }
}
```

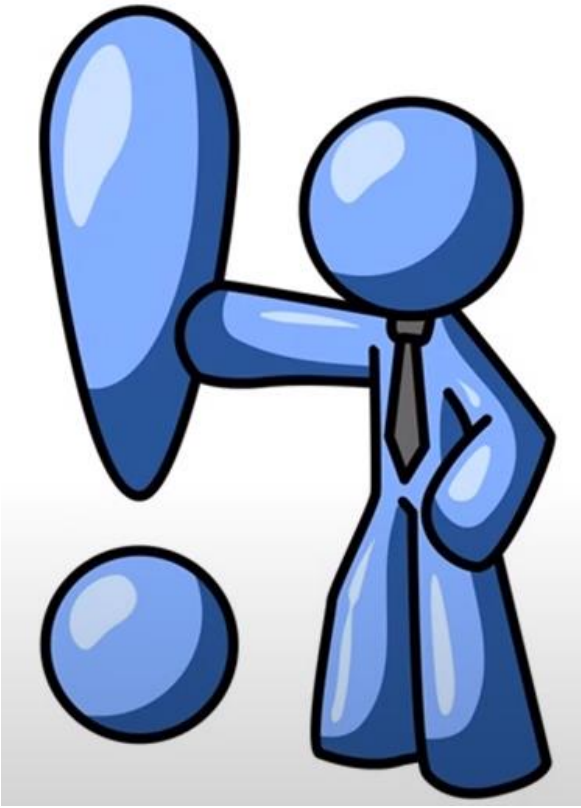
```
class NotificadorSMS implements
Notificador {
    @Override
    public void enviarNotificacion(String
mensaje) {
        // Enviar notificación por SMS
    }
}
```

De esta forma, podemos añadir nuevos canales de notificación creando nuevas clases que implementen la interfaz Notificador, sin modificar las clases existentes.

CONCLUSIONES

Principio de Sustitución de Liskov (LSP): El LSP establece que las subclases deben poder reemplazar a las clases base sin alterar el funcionamiento del programa. Esto asegura la mantenibilidad y reutilización del código, evitando errores al sustituir subclases por clases base.

Principio de Abierto/Cerrado (OCP): El OCP dicta que el software debe estar abierto para la extensión, pero cerrado para la modificación. Esto permite agregar nuevas funcionalidades extendiendo las clases existentes sin modificar su código original, facilitando la evolución y estabilidad del software.



Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

METACOGNICIÓN

1 ¿ QUÉ HE APRENDIDO ?

SER CONSCIENTE

2 ¿ CÓMO LO HE APRENDIDO ?

DESCRIBIR

3 ¿ QUÉ HA RESULTADO MÁS FÁCIL, MÁS DIFICIL O MÁS NOVEDOSO ?

EVALUAR

4 ¿ CÓMO LO PUEDO MEJORAR? ¿PARA QUÉ ME HA SERVIDO?

PLANIFICAR

Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

¡GRACIAS!

EN LA UTP SABEMOS QUE TODO ES POSIBLE